02246 Mandatory Assignment
Assignment 04 - Probabilistic Model Checking *

To be submitted on DTU Learn - see deadline on DTU Learn

You are encouraged to work in groups, but you must clearly identify the contributions of each group member, and you will be jointly responsible for the finished report. Register your group on DTU Learn before submitting as group submission.

Answers to all parts should be typed up using LaTeX and submitted electronically as a PDF report using the provided template. Drawings and formulae may be handwritten and scanned. More detailed instructions as to the style of answer we expect for each part are included below.

Some tasks require to upload files.

*Thanks to Michael Smith (the original author), and Lijun Zhang, Kebin Zeng, Flemming Nielson, Alberto Lluch Lafuente and Andrea Vandin (contributors).

Assignment 04 - Probabilistic Model Checking

A04P: Practical Problems

A04P.1 - Stefano Giroletti *Lottery scheduling* is a scheduling discipline where each task is assigned a number of tickets. To decide which task to execute, the scheduler randomly selects a ticket (under a uniform distribution), and the task owning the ticket is allowed to execute. A lottery scheduler can be either *pre-emptive*, in which a task can only execute for a certain amount of time before being interrupted, or *non pre-emptive* if a task always executes to completion once selected.

a) Construct a PRISM model of a pre-emptive lottery scheduler with three clients, where each task is given a single ticket, and the quantum (the length of time before a task is interrupted) is one time unit. You will have to decide on an appropriate way to model the tickets, which means deciding on an appropriate level of abstraction.

Modelling Choices for the Pre-emptive Lottery Scheduler

- **Ticket Assignment:** At job creation, the scheduler deterministically assigns 1 ticket to the new job. This explicit step makes the model flexible for different assignment methods in following exercises.
- **Job Length:** Upon creation, each job randomly and uniformly selects an execution length between 1 and 5 time units.
- **Execution Logic (1-unit quantum):** Jobs with at least one ticket are eligible to run. The CPU is granted to the job with the highest ticket count. For three jobs (j_1, j_2, j_3), the scheduling guards are:
  - $tickets(j_1) \geq tickets(j_2) \wedge tickets(j_1) \geq tickets(j_3) \rightarrow j_1$ executes.
  - $tickets(j_2) \geq tickets(j_1) \wedge tickets(j_2) \geq tickets(j_3) \rightarrow j_2$ executes.
  - $tickets(j_3) \geq tickets(j_1) \wedge tickets(j_3) \geq tickets(j_2) \rightarrow j_3$ executes.
- **Non-Determinism and Ties:** Because jobs can have equal tickets, guards overlap. This non-determinism is naturally resolved by PRISM's underlying uniform distribution, which simulates a fair lottery draw among the tied jobs.

UPLOAD REQUIRED: a prism model file `A04P.1.a.prism`.

b) We are now interested in the time taken for the first task from $\texttt{Client}_1$ to complete. To do so, let's introduce a new module called `Monitor`, which has a single Boolean variable called `finished`. The module should only perform $\texttt{finish}_1$ actions — the first time it does a $\texttt{finish}_1$ it changes its state so that `finished` = `true`, and for subsequent actions it remains in this state.

UPLOAD REQUIRED: a prism model file `A04P.1.b.prism`.

c) Write down a PCTL formula (using the '`P=?`' syntax), expressing the probability that the first task from $\texttt{Client}_1$ completes within t time units — you should define t as an integer constant, without a value. Use the experimentation feature of PRISM (as described in the tutorial), to plot a graph of this probability, for $0 \leq t \leq 20$.

The required PCTL formula is:

```
P=?  [F<=t (finished=true)]
```

The corresponding plot for $0 \leq t \leq 20$ is shown in Figure 1.

UPLOAD REQUIRED: a prism property file `A04P.1.c.props`.

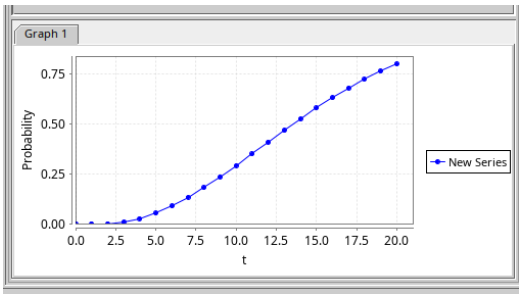


Figure 1: Probability for job 1 to complete within t time units.

d) Is it possible for your lottery scheduler to suffer from starvation?

Theoretically speaking, starvation is possible, which can be verified by the formula `A [ F (finished=true) ]` that PRISM evaluates as false in the model. Nevertheless, this path has probability 0; in fact, the query `P=? [ F (finished=true) ]` returns 1, meaning that a job **almost surely terminates**.

e) Modify your model so that the quantum is two time units. How does this affect the time taken to complete a task from $\texttt{Client}_1$?

In order to check how the time changes with respect to the 1 time quantum scheduler we plot for both models the graph of P=? [F $\leq$ t(finished=true)]. As we can see from the graph for a given t the probability for job1 to finih before t are higher for the scheduler with 2 time quantum (green line)

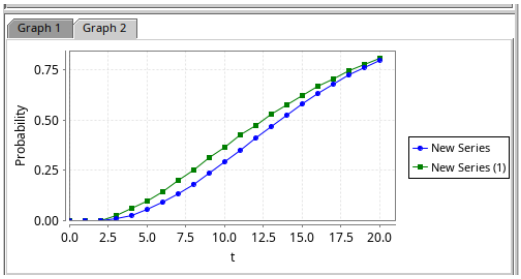


Figure 2: Compared probability in scheduler 1 and scheduler 2 for job 1 to complete within t time units.

UPLOAD REQUIRED: a prism model file `A04P.1.e.prism`.

A04P.2 - Loke S. G. Jørgensen - Stefano Giroletti Rather than assigning the same number of tickets to each task, we can give varying priority to different tasks by assigning them *different* numbers of tickets.

a) By assigning different numbers of tickets to different tasks, approximate an SRT scheduler using lottery scheduling.

The number of tickets can now range from 0 (indicating no job is ready) to 5, and they are assigned with uniform probability upon job creation.

UPLOAD REQUIRED: a prism model file `A04P.2.a.prism`.

b) How does this affect the time taken to complete a task from $\texttt{Client}_1$? Is starvation possible?

Regarding starvation, the situation remains unchanged: the formula `A [ F (finished=true) ]` still evaluates to false, confirming the theoretical possibility of starvation, while the query `P=? [ F (finished=true) ]` still returns 1, ensuring almost sure termination.

The impact on execution time is illustrated in Figure 3, which compares the probability of Job 1 finishing within t time units under the original 1-ticket scheduler versus the new multi-ticket scheduler. As the graph shows, the up-to-5 tickets scheduler (green line) yields a higher completion probability in the first few steps. However, in the long run, the 1-ticket scheduler (blue line) reaches a higher probability sooner; because every job has exactly 1 ticket, it is easier for Job 1 to be repeatedly selected without having to constantly compete against jobs holding a permanently higher ticket count.

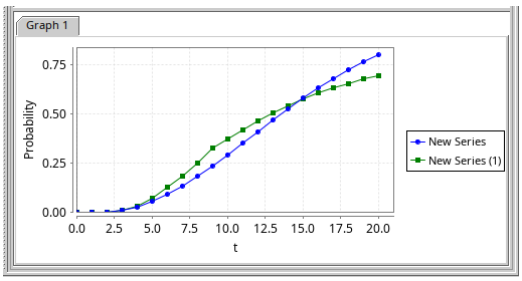


Figure 3: Comparison of the probability for Job 1 to complete within t time units between the 1-ticket and the up-to-5 tickets schedulers.

c) Increase the number of clients in your model until you reach the maximum for which PRISM can still build the underlying DTMC. Compare the time to build the DTMC to the time to run some analysis on it (e.g. a PCTL property).

PRISM encounters an "out of memory" error when attempting to build the DTMC with 9 clients (expanding the model `A04.P.2.a.prism`). With 8 clients, the DTMC is successfully constructed in 12.83 seconds, as shown in Figure 4.

However, during property verification, both the 7-client and 8-client models run out of memory. Consequently, we scaled down the experiments to 6 clients, which requires a significantly shorter build time of 0.337 seconds. For this 6-client model, the query `P=? [ F (finished=true) ]` is evaluated in 41.99 seconds, while the query `E [ F (finished=true) ]` takes 16.305 seconds to evaluate.

```
---------------------------------------------------------------------

Building model (engine:symbolic)...

Computing reachable states...

Reachability (BFS): 20 iterations in 0.72 seconds (average 0.036000, setup 0.00)

Time for model construction: 12.83 seconds.

Type:        DTMC
States:      1705782074882 (1 initial)
Transitions: 15971940656080

Transition matrix: 4069409 nodes (16 terminal), 15971940656080 minterms, vars: 57r/57c
```

Figure 4: Model building time for 8 clients.

d) Investigate what happens if one client tries to perform long jobs, but the others all generate very short jobs. Is the scheduler vulnerable to a denial of service attack?

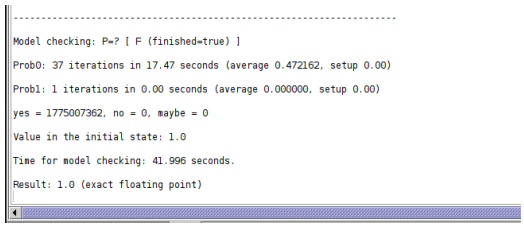


Figure 5: Evaluation time for `P=? [ F (finished=true) ]` with 6 clients.

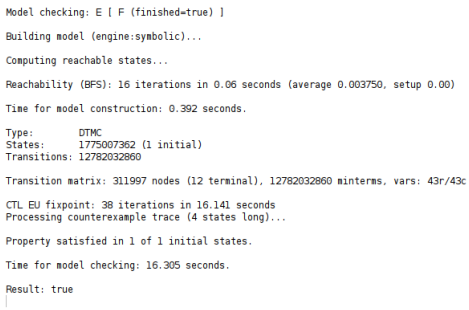


Figure 6: Evaluation time for `A [ F (finished=true) ]` with 6 clients.

To investigate the behavior, I modified the model such that client 3 always generates jobs with length 5, and extended the monitor with `finished` variables for all three clients. I then queried `P=? [ F finished ]` and `P=? [ F<=20 finished ]` for each of the three clients, and found that clients 1 and 2 finished their jobs within 20 iterations with probability P=0.72, and finished eventually with P=1.0. An adversary who repeatedly schedules long jobs could certainly cause delays, but they could not starve the other clients.

e) Can you express denial of service as a PCTL property? Explain your answer.

I would use the properties I described in my answer to the previous question: `P=? [ F finished ]` to investigate whether starvation is possible, and `P=? [ F<=t finished ]` for an integer constant t to investigate how much of a delay an adversary could cause.

A04P.3 - Loke S. G. Jørgensen In A04P.2 we reached the limits of model checking—here we explore the obscure abyss beyond them by resorting to simulation.

a) Replicate the model from A04P.2 but defining 10 clients, each with tasks of size six (i.e. of type `[0..5]`), and each also having six tickets defined in the same manner. Define a monitor that synchronises with these clients, and lets us observe when *all clients* have finished their tasks. Check how, in its default configuration, PRISM mems-out when trying to build this model. Report the *(i)* runtime and *(ii)* max memory used to reach this failure.

Using linux diagnostics, the new PRISM model crashed after 28.10 seconds with 1.29 gigabytes of memory used.

UPLOAD REQUIRED: a prism model file `A04P.3.a.prism`.

b) Define a PCTL property using `P=?`, that queries the probability with which all clients may finish their tasks within t time units (the same integer constant defined for

A04P.1). Estimate this probability for $t \in \{50, 56, 58, 60, 70, 80, 90, 100\}$ using the SMC capabilities of PRISM in their default configuration, as shown in class. Besides the estimated probabilities, report the width of the confidence interval achieved, and the runtime used to compute this estimate.

I used a formula `all_finished = finished1 & finished2 & ... finished10` to build a PCTL property `P=? [ F<=t all_finished ]`.

Table 1: SMC estimates for the probability that all clients finish within t steps.

t	Estimated probability	99% CI width	Runtime (s)
50	0.000	0.000000	0.700
56	0.000	0.000000	0.798
58	0.000	0.000000	0.757
60	0.000	0.000000	0.809
70	0.000	0.000000	0.826
80	0.002	0.007282	0.891
90	0.005	0.011496	0.980
100	0.009	0.015393	1.072

c) Repeat the previous exercise with 10^4 simulations—in the modal dialog that opens when we simulate a property, in the field "Number of samples"—and also 10^5.

Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

d) Using simulation as above, find the probability with which all clients may finish their tasks within $t = 56$ time units (hint: it ain't zero), producing a confidence interval that does not contain zero.

Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

A04T: Theoretical Problems

A04T.1 - Maria Mitsi Consider the DTMC in Figure 7(a), where the initial state is s_0, and the labels are shown on the states.

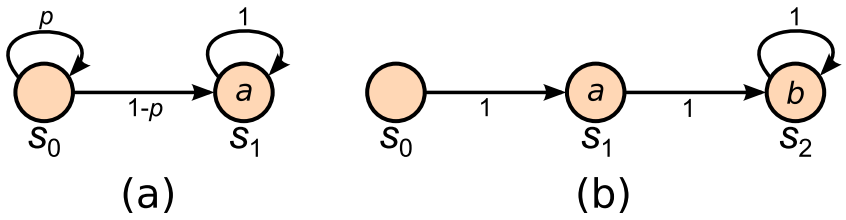


Figure 7: More DTMCs

a) For what values of p is it the case that $s_0 \models \mathbb{P}_{\geq \frac{1}{2}}(F^{\leq 2}\ a)$?

We want to calculate the probability of reaching a state labelled a within at most 2 transitions, starting from s_0.

The only state labelled a is s_1, which is reached from s_0 with probability $1-p$. State s_0 has a self-loop with probability p, while s_1 has a self-loop with probability 1.

There are two possible ways to reach s_1 within 2 steps:

- Reach s_1 in the first transition, with probability $1-p$.
- Stay in s_0 in the first transition and reach s_1 in the second transition, with probability $p(1-p)$.

Therefore, the probability of reaching a within 2 steps is

$$\begin{aligned} P(F^{\leq 2}\ a) &= (1-p) + p(1-p) \\ &= 1 - p + p - p^2 \\ &= 1 - p^2. \end{aligned}$$

We need this probability to be at least $\frac{1}{2}$:

$$\begin{aligned} 1 - p^2 &\geq \frac{1}{2} \\ p^2 &\leq \frac{1}{2} \\ p &\leq \frac{1}{\sqrt{2}}. \end{aligned}$$

Since p is a probability, $0 \leq p \leq 1$.

Thus, the required values are

$$\boxed{0 \leq p \leq \frac{1}{\sqrt{2}}}.$$

A04T.2 - Maria Mitsi Consider the DTMC in Figure 8, which we will call $\mathcal{M} = (S, \boldsymbol{P}, s_1, AP, L)$, where $AP = \{b, c\}$. The initial state is s_1 and the labels are shown on the states. Determine which states of the DTMC satisfy the following PCTL formulae, by following the PCTL model checking algorithm seen in the course:

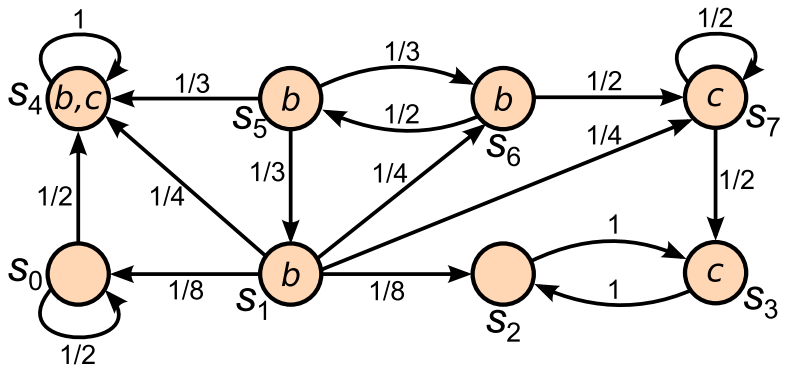


Figure 8: Another DTMC

a) $\mathbb{P}_{\geq \frac{17}{19}}(b\ U\ c)$

This formula asks for the states from which the probability of reaching a state labelled c, while remaining in states labelled b until then, is at least $\frac{17}{19}$.

First, the states labelled c are

$$Sat(c) = \{s_3, s_4, s_7\}.$$

The states labelled b are

$$Sat(b) = \{s_1, s_4, s_5, s_6\}.$$

Since c is already true in s_3, s_4 and s_7, the probability of satisfying the formula from these states is 1.

For the remaining states, we calculate the probabilities of reaching c while staying in b.

Let x_i denote the probability of satisfying $b\ U\ c$ starting from state s_i.

From the transitions in the diagram, we obtain:

$$x_1 = \frac{1}{4} + \frac{1}{4} + \frac{1}{4}x_6$$
$$x_5 = \frac{1}{3} + \frac{1}{3}x_1 + \frac{1}{3}x_6$$
$$x_6 = \frac{1}{2} + \frac{1}{2}x_5.$$

Simplifying the first equation:

$$x_1 = \frac{1}{2} + \frac{1}{4}x_6.$$

Substituting this into the other equations:

$$x_5 = \frac{1}{3} + \frac{1}{3}\left(\frac{1}{2} + \frac{1}{4}x_6\right) + \frac{1}{3}x_6.$$

Therefore,

$$x_5 = \frac{1}{2} + \frac{5}{12}x_6.$$

Using $x_6 = \frac{1}{2} + \frac{1}{2}x_5$, we get

$$x_5 = \frac{1}{2} + \frac{5}{12}\left(\frac{1}{2} + \frac{1}{2}x_5\right).$$

Solving:

$$x_5 = \frac{17}{19}.$$

Consequently,

$$x_6 = \frac{1}{2} + \frac{1}{2} \cdot \frac{17}{19} = \frac{18}{19},$$

and

$$x_1 = \frac{1}{2} + \frac{1}{4} \cdot \frac{18}{19} = \frac{14}{19}.$$

We now compare the probabilities with the required threshold $\frac{17}{19}$:

State	Probability	Satisfies?
s_0	0	No
s_1	$\frac{14}{19}$	No
s_2	0	No
s_3	1	Yes
s_4	1	Yes
s_5	$\frac{17}{19}$	Yes
s_6	$\frac{18}{19}$	Yes
s_7	1	Yes

Therefore, the set of satisfying states is

$$\boxed{Sat\left(\mathbb{P}_{\geq \frac{17}{19}}(b\ U\ c)\right) = \{s_3, s_4, s_5, s_6, s_7\}.}$$

b) $\mathbb{P}_{\geq \frac{1}{2}}(X\ \mathbb{P}_{> \frac{1}{3}}((b \vee c)\ U^{\leq 2}\ (b \wedge c)))$

First, we identify the states satisfying $b \wedge c$. Only s_4 has both labels:

$$Sat(b \wedge c) = \{s_4\}.$$

The states satisfying $b \vee c$ are

$$Sat(b \vee c) = \{s_1, s_3, s_4, s_5, s_6, s_7\}.$$

We calculate the probability of reaching s_4 within at most 2 transitions, while remaining in states satisfying $b \vee c$ until reaching s_4.

Let y_i denote this probability when starting from s_i.

Since s_4 already satisfies $b \wedge c$:

$$y_4 = 1.$$

For s_5, there are two ways to reach s_4 within 2 transitions:

- Move directly to s_4, with probability $\frac{1}{3}$.
- Move to s_1 and then to s_4, with probability $\frac{1}{3} \cdot \frac{1}{4}$.

Thus,

$$y_5 = \frac{1}{3} + \frac{1}{3} \cdot \frac{1}{4} = \frac{5}{12}.$$

For s_6, the only way to reach s_4 within 2 transitions is to move to s_5 and then to s_4:

$$y_6 = \frac{1}{2} \cdot \frac{1}{3} = \frac{1}{6}.$$

From s_1, the probability of reaching s_4 directly is $\frac{1}{4}$. No other successor can reach s_4 in the remaining one transition. Therefore,

$$y_1 = \frac{1}{4}.$$

From s_0, s_2, s_3 and s_7, the probability is zero, because s_4 cannot be reached within the required number of transitions while satisfying the condition.
We obtain the following probabilities:

State	y_i
s_0	0
s_1	$\frac{1}{4}$
s_2	0
s_3	0
s_4	1
s_5	$\frac{5}{12}$
s_6	$\frac{1}{6}$
s_7	0

The inner probability must be strictly greater than $\frac{1}{3}$. Therefore, the states satisfying the inner formula are

$$\boxed{Sat\left(\mathbb{P}_{>\frac{1}{3}}\left((b \vee c)\ U^{\leq 2}\ (b \wedge c)\right)\right) = \{s_4, s_5\}.}$$

We now evaluate

$$\mathbb{P}_{\geq \frac{1}{2}}(X\ \varphi),$$

where

$$Sat(\varphi) = \{s_4, s_5\}.$$

The next-state operator X means that we only consider the probability of moving to one of these two states in the next transition.
We calculate this probability for each state:

State	$P(\text{next state} \in \{s_4, s_5\})$	Satisfies?
s_0	$\frac{1}{2}$	Yes
s_1	$\frac{1}{4}$	No
s_2	0	No
s_3	0	No
s_4	1	Yes
s_5	$\frac{1}{3}$	No
s_6	$\frac{1}{2}$	Yes
s_7	0	No

We include states where the probability is equal to $\frac{1}{2}$ because the inequality is $\geq$. Therefore, the final set of satisfying states is

$$\boxed{Sat\left(\mathbb{P}_{\geq\frac{1}{2}}\left(X\ \mathbb{P}_{>\frac{1}{3}}\left((b\vee c)\ U^{\leq 2}\ (b\wedge c)\right)\right)\right)=\{s_0,s_4,s_6\}.}$$

A04T.3 - Mithra Iyengar Ramprabu As we have seen in the course, a step-bounded until formula $\Phi_1\ U^{\leq n}\ \Phi_2$ in PCTL is satisfied by a path if Φ_2 holds after at most n time steps, and until then Φ_1 always holds. We would like to generalise this to the form $\Phi_1\ U^I\ \Phi_2$, where I is an interval over the natural numbers.

a) Define the semantics $\mathcal{M},\sigma\models\Phi_1\ U^I\ \Phi_2$ of this new path formula, where σ is a path in the DTMC $\mathcal{M}$. You have some freedom to define the semantics, and there are several meaningful options.

For a path $\sigma=s_0s_1s_2\ldots$ of the DTMC $\mathcal{M}$:

$$\mathcal{M},\sigma\models\Phi_1\ U^I\ \Phi_2\iff\exists k\in I:\ s_k\models\Phi_2\ \text{ and }\ \forall j<k:\ s_j\models\Phi_1.$$

b) Consider four cases on the form of the interval I, for $0<n\leq n'$: $[0,\infty)$, $[0,n]$, $[n,\infty)$ and $[n,n']$. For each case, can you encode the formula $\mathbb{P}_J(\Phi_1\ U^I\ \Phi_2)$ in terms of the existing PCTL operators? If so, show how, and if not, explain why.

- $[0,\infty)$: yes, $\mathbb{P}_J(\Phi_1\ U\ \Phi_2)$.
- $[0,n]$: yes, $\mathbb{P}_J(\Phi_1\ U^{\leq n}\ \Phi_2)$.
- $[n,\infty)$ and $[n,n']$: no. For $n=1$ the probability from s is $\sum_{s'}P(s,s')\cdot\Pr_{s'}(\Phi_1\ U\ \Phi_2)$, a weighted sum of the successors' exact reachability probabilities. PCTL nests subformulas only through their satisfaction sets (true/false labels), so it loses these values. Also, $\Phi_1\wedge X(\Phi_1\ U\ \Phi_2)$ is not a legal PCTL path formula, since X, U and $U^{\leq n}$ may only be used directly under $\mathbb{P}$.

c) For the cases where you cannot encode the formula, could you encode them in PTCL*?

Yes. A witness $k\geq n$ requires Φ_1 at steps $0,\ldots,n-1$ and $\Phi_1\ U\ \Phi_2$ from step n on, so

$$[n,\infty):\quad \mathbb{P}_J\big(\Phi_1\wedge X(\Phi_1\wedge\cdots\wedge X(\Phi_1\ U\ \Phi_2)\ldots)\big)\quad (n\text{ copies of }\Phi_1,\ n\text{ nested }X),$$
$$[n,n']:\quad \text{the same, ending with }\Phi_1\ U^{\leq n'-n}\ \Phi_2.$$

PCTL* allows this because it permits arbitrary nesting of X, U and Boolean connectives under $\mathbb{P}$.

d) Design a model checking algorithm for $\mathbb{P}_J(\Phi_1\ U^{\geq n}\ \Phi_2)$, i.e. $\mathbb{P}_J(\Phi_1\ U^{[n,\infty)}\ \Phi_2)$. Get inspiration from the algorithms seen in the course for determining $\mathbb{P}_J(\Phi_1\ U\ \Phi_2)$. Explain the correctness of your proposed algorithm.

Algorithm.

(a) Compute $x(s)=\Pr_s(\Phi_1\ U\ \Phi_2)$ for all states with the standard algorithm.

(b) Set $y^{(0)} = x$. For $i = 0, \ldots, n-1$ and every state s:

$$y^{(i+1)}(s) = \begin{cases} \sum_{s'} P(s,s')\, y^{(i)}(s') & \text{if } s \models \Phi_1, \\ 0 & \text{otherwise.} \end{cases}$$

(c) Return $\{\, s \mid y^{(n)}(s) \in J \,\}$.

Correctness: By induction on i, $y^{(i)}(s)$ is the probability, starting in s, that Φ_1 holds at steps $0, \ldots, i-1$ and the suffix from step i satisfies $\Phi_1\ U\ \Phi_2$. For $i = 0$ this is the standard algorithm. For the step $i \to i+1$, condition on the first successor: if $s \not\models \Phi_1$ the set of paths is empty, otherwise the probability is $\sum_{s'} P(s,s')\, y^{(i)}(s')$. For $i = n$ this is exactly $\Phi_1\ U^{[n,\infty)}\ \Phi_2$, because a witness $k \geq n$ needs Φ_1 on steps $0, \ldots, n-1$ and $\Phi_1\ U\ \Phi_2$ from step n.

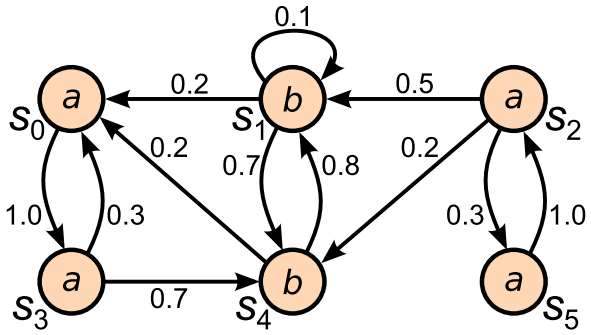


Figure 9: One more DTMC